SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: MLA03
COURSE CODE: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems.(BL2, BL3)*

Assessment Tool Weightage: 30%

Dr G.A. SENTHIL

QUESTIONS: 05 **Total Marks:50**

Annexure A

Constraint - Based Problem-Solving **Duration: 2Hrs**

1.Implement an ε-greedy Multi-Armed Bandit algorithm in Python to maximize rewards under a limited budget of 500 trials. Compare the effect of different ε values (0.1, 0.3, and 0.5) on exploration, exploitation, and constraint satisfaction.

CODE :

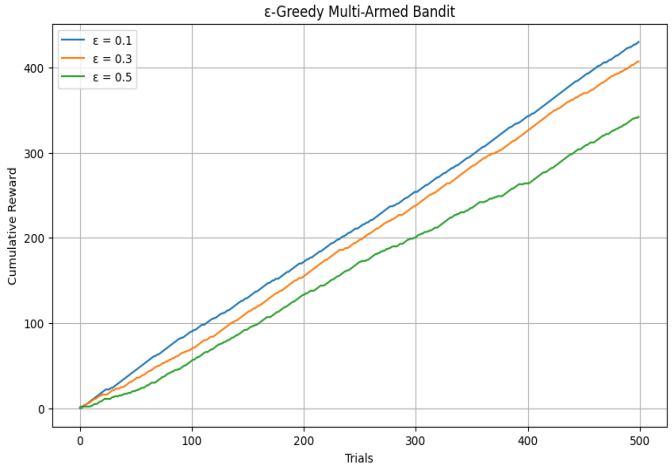
```
import numpy as np
import matplotlib.pyplot as plt

# Make the output reproducible
np.random.seed(42)

# True reward probabilities of 5 slot machines (arms)
true_probabilities = [0.2, 0.5, 0.7, 0.4, 0.9]

num_arms = len(true_probabilities)
trials = 500

# ----------------------------------------------------
# ε-Greedy Algorithm
# ----------------------------------------------------
def epsilon_greedy(epsilon):
```

```
    estimated_rewards = np.zeros(num_arms)
    arm_counts = np.zeros(num_arms)

    total_reward = 0
    cumulative_rewards = []

    exploration = 0
    exploitation = 0

    for t in range(trials):

        # Exploration
        if np.random.rand() < epsilon:
            arm = np.random.randint(num_arms)
            exploration += 1

        # Exploitation
        else:
            arm = np.argmax(estimated_rewards)
            exploitation += 1

        # Reward (1 = Success, 0 = Failure)
        reward = 1 if np.random.rand() < true_probabilities[arm] else 0

        arm_counts[arm] += 1

        # Update estimated reward
        estimated_rewards[arm] += (reward - estimated_rewards[arm]) / arm_counts[arm]

        total_reward += reward
        cumulative_rewards.append(total_reward)

    return cumulative_rewards, total_reward, exploration, exploitation


epsilons = [0.1, 0.3, 0.5]

plt.figure(figsize=(10,6))

print("Results after 500 Trials\n")

for eps in epsilons:

    rewards, total, explore, exploit = epsilon_greedy(eps)

    print("-------------------------------------")
    print(f"Epsilon = {eps}")
    print(f"Total Reward      : {total}")
    print(f"Exploration Steps  : {explore}")
    print(f"Exploitation Steps : {exploit}")
```

```
    if explore + exploit == trials:
        print("Constraint (500 Trials): Satisfied")

    plt.plot(rewards, label=f"ε = {eps}")

plt.title("ε-Greedy Multi-Armed Bandit")
plt.xlabel("Trials")
plt.ylabel("Cumulative Reward")
plt.legend()
plt.grid(True)
plt.show()
```

OUTPUT:

```
Results after 500 Trials

-------------------------------------
Epsilon = 0.1
Total Reward       : 430
Exploration Steps  : 43
Exploitation Steps : 457
Constraint (500 Trials): Satisfied
-------------------------------------
Epsilon = 0.3
Total Reward       : 407
Exploration Steps  : 142
Exploitation Steps : 358
Constraint (500 Trials): Satisfied
-------------------------------------
Epsilon = 0.5
Total Reward       : 342
Exploration Steps  : 259
Exploitation Steps : 241
Constraint (500 Trials): Satisfied
```



2.Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption. Define the states,actions, transition probabilities, rewards, and energy constraints.

CODE

```
import numpy as np

# Grid size
GRID_SIZE = 4

# Start and Goal positions
start = (0, 0)
goal = (3, 3)

# Obstacles
obstacles = [(1, 1), (2, 2)]

# Battery energy
battery = 20

# Actions
```

```
actions = {
    0: (-1, 0),   # Up
    1: (1, 0),    # Down
    2: (0, -1),   # Left
    3: (0, 1)     # Right
}

state = start
total_reward = 0

print("Autonomous Robot MDP Simulation")
print("--------------------------------")

while battery > 0:

    print(f"\nCurrent State : {state}")
    print(f"Battery       : {battery}")

    # Stop if goal reached
    if state == goal:
        print("Goal Reached!")
        total_reward += 100
        break

    # Random action (MDP transition)
    action = np.random.choice([0, 1, 2, 3])

    move = actions[action]

    next_state = (
        max(0, min(GRID_SIZE - 1, state[0] + move[0])),
        max(0, min(GRID_SIZE - 1, state[1] + move[1]))
    )

    # Transition Probability
    transition_probability = 0.9

    # Check obstacle
    if next_state in obstacles:
        reward = -20
        battery -= 5
        print("Obstacle Hit!")
    else:
        reward = -1
        battery -= 2
        state = next_state

    total_reward += reward

print("\n---------------------------")
```

```
print("Final State :", state)
print("Remaining Battery :", battery)
print("Total Reward :", total_reward)
```

OUTPUT:

```
Autonomous Robot MDP Simulation
--------------------------------

Current State : (0, 0)
Battery       : 20

Current State : (1, 0)
Battery       : 18

Current State : (0, 0)
Battery       : 16

Current State : (0, 0)
Battery       : 14

Current State : (1, 0)
Battery       : 12

Current State : (2, 0)
Battery       : 10

Current State : (3, 0)
Battery       : 8

Current State : (3, 1)
Battery       : 6

Current State : (3, 1)
Battery       : 4

Current State : (2, 1)
Battery       : 2
Obstacle Hit!

---------------------------
Final State : (2, 1)
Remaining Battery : -3
Total Reward : -29
```

3.A warehouse robot has a battery capacity of only 30 minutes. Design an RL framework that enables the robot to complete the maximum number of deliveries without exceeding the battery limit. Explain the constraints, policy, and reward design.

CODE:

```
import random



# Battery capacity (minutes)

battery = 30
```

```
# Initial values
deliveries = 0
total_reward = 0

print("Warehouse Robot Reinforcement Learning")
print("--------------------------------------")
print("Initial Battery:", battery, "minutes\n")

while battery > 0:

    # Random delivery time between 3 and 7 minutes
    delivery_time = random.randint(3, 7)

    # Stop if battery is insufficient
    if battery < delivery_time:
        print("Battery too low for next delivery.")
        break

    # Perform delivery
    battery -= delivery_time
    deliveries += 1
    reward = 10
    total_reward += reward

    print(f"Delivery {deliveries}")
    print(f"Time Used       : {delivery_time} minutes")
    print(f"Remaining Battery: {battery} minutes")
```

```
    print(f"Reward          : +{reward}")
    print()


print("--------------------------------------")
print("Total Deliveries :", deliveries)
print("Remaining Battery:", battery, "minutes")
print("Total Reward     :", total_reward)
```

OUTPUT:

```
Warehouse Robot Reinforcement Learning
--------------------------------------
Initial Battery: 30 minutes

Delivery 1
Time Used        : 7 minutes
Remaining Battery: 23 minutes
Reward           : +10

Delivery 2
Time Used        : 6 minutes
Remaining Battery: 17 minutes
Reward           : +10

Delivery 3
Time Used        : 5 minutes
Remaining Battery: 12 minutes
Reward           : +10

Delivery 4
Time Used        : 7 minutes
Remaining Battery: 5 minutes
Reward           : +10

Battery too low for next delivery.
--------------------------------------
Total Deliveries : 4
Remaining Battery: 5 minutes
Total Reward     : 40
```

4.Analyse an experimental comparison between random action selection and ε-greedy action selection in a Multi-Armed Bandit problem under a fixed time constraint. Record cumulative rewards and explain which strategy provides better performance.

CODE:

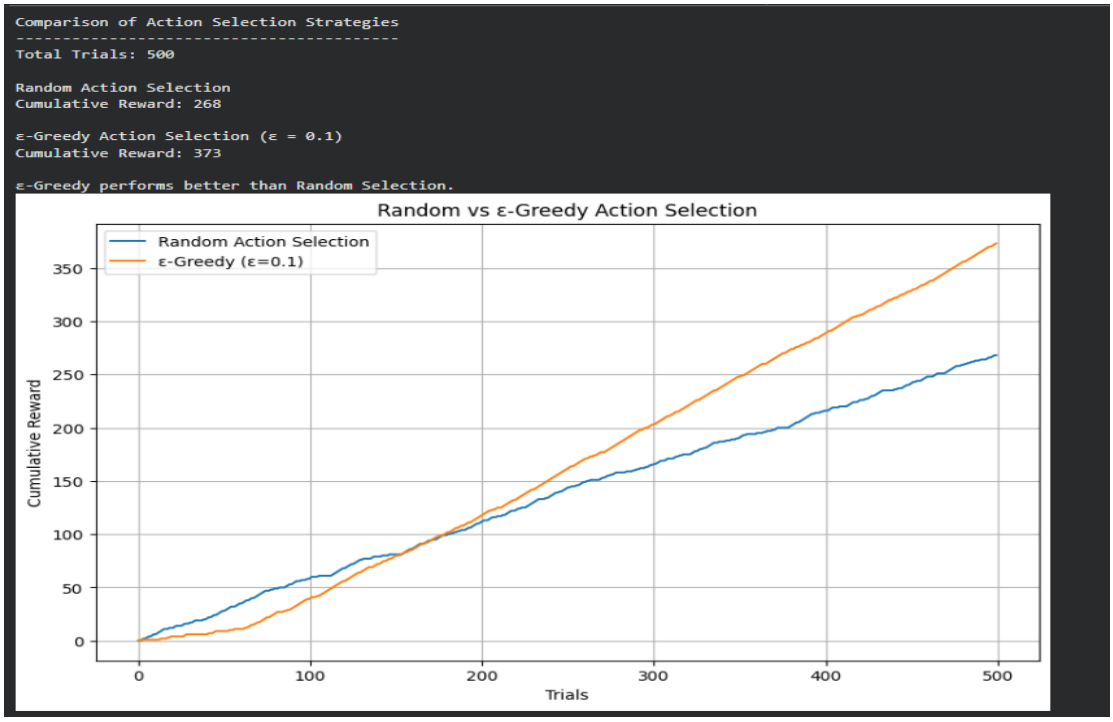
```
import numpy as np
import matplotlib.pyplot as plt

# Make results reproducible
np.random.seed(42)

# Reward probabilities for 5 slot machines
true_rewards = [0.2, 0.5, 0.7, 0.4, 0.9]

num_arms = len(true_rewards)
trials = 500

# ----------------------------
# Random Action Selection
# ----------------------------
def random_selection():

    total_reward = 0
    cumulative_rewards = []

    for _ in range(trials):
        arm = np.random.randint(num_arms)

        reward = 1 if np.random.rand() < true_rewards[arm] else 0

        total_reward += reward
```

```
        cumulative_rewards.append(total_reward)

    return cumulative_rewards, total_reward



# ----------------------------
# ε-Greedy Action Selection
# ----------------------------
def epsilon_greedy(epsilon=0.1):

    estimated_rewards = np.zeros(num_arms)
    arm_counts = np.zeros(num_arms)

    total_reward = 0
    cumulative_rewards = []

    for _ in range(trials):

        if np.random.rand() < epsilon:
            arm = np.random.randint(num_arms)      # Explore
        else:
            arm = np.argmax(estimated_rewards)     # Exploit

        reward = 1 if np.random.rand() < true_rewards[arm] else 0

        arm_counts[arm] += 1
```

```
        estimated_rewards[arm] += (reward - estimated_rewards[arm]) / arm_counts[arm]

        total_reward += reward
        cumulative_rewards.append(total_reward)

    return cumulative_rewards, total_reward


# Run both strategies
random_rewards, random_total = random_selection()
greedy_rewards, greedy_total = epsilon_greedy(0.1)

# Print Results
print("Comparison of Action Selection Strategies")
print("----------------------------------------")
print("Total Trials:", trials)
print()
print("Random Action Selection")
print("Cumulative Reward:", random_total)
print()
print("ε-Greedy Action Selection (ε = 0.1)")
print("Cumulative Reward:", greedy_total)

if greedy_total > random_total:
    print("\nε-Greedy performs better than Random Selection.")
else:
    print("\nRandom Selection performs better.")
```

```
# Plot graph
plt.figure(figsize=(10,6))
plt.plot(random_rewards, label="Random Action Selection")
plt.plot(greedy_rewards, label="ε-Greedy (ε=0.1)")
plt.xlabel("Trials")
plt.ylabel("Cumulative Reward")
plt.title("Random vs ε-Greedy Action Selection")
plt.legend()
plt.grid(True)
plt.show()
```

OUTPUT:



5.Implement a Reinforcement Learning solution using Python for a smart energy management system where electricity consumption must remain below a specified threshold. Explain how reward shaping, policy learning, and feasibility constraints improve decision-making.

CODE:

```
import random

# Threshold for electricity consumption (units)
THRESHOLD = 100

# Initial values
current_consumption = 0
total_reward = 0

# Appliances and their energy consumption
appliances = {
    "Light": 10,
    "Fan": 20,
    "AC": 40,
    "Washing Machine": 30,
    "Refrigerator": 25
}

print("Smart Energy Management using Reinforcement Learning")
print("----------------------------------------------------")
print("Maximum Allowed Consumption:", THRESHOLD, "Units\n")

while True:

    appliance = random.choice(list(appliances.keys()))
    energy = appliances[appliance]
```

```
    # Check feasibility constraint
    if current_consumption + energy <= THRESHOLD:

        current_consumption += energy
        reward = 10
        total_reward += reward

        print(f"{appliance} ON")
        print(f"Energy Used        : {energy} Units")
        print(f"Total Consumption  : {current_consumption}")
        print(f"Reward             : +{reward}\n")

    else:

        print(f"{appliance} cannot be turned ON")
        print("Threshold Exceeded!")
        reward = -20
        total_reward += reward
        break

print("-----------------------------------------")
print("Final Electricity Consumption :", current_consumption)
print("Total Reward                  :", total_reward)
```

OUTPUT:

```
Smart Energy Management using Reinforcement Learning
-----------------------------------------------------
Maximum Allowed Consumption: 100 Units

Refrigerator ON
Energy Used        : 25 Units
Total Consumption  : 25
Reward             : +10

AC ON
Energy Used        : 40 Units
Total Consumption  : 65
Reward             : +10

Washing Machine ON
Energy Used        : 30 Units
Total Consumption  : 95
Reward             : +10

AC cannot be turned ON
Threshold Exceeded!
-------------------------------------------
Final Electricity Consumption : 95
Total Reward                  : 10
```